MODULE 01

SYLLABUS: Arduino Basics: Arduino Uno R3 components, Microcontrollers in other Arduinos, Programming Interfaces, Input/Output: GPIO, ADCs, and Communication Busses, Bootloaders and Firmware. Digital Inputs, Outputs and PWM: Digital Outputs, Controlling an LED on a breadboard with digitalWrite(), Controlling an RGB LED, Pulse-Width Modulation with analogWrite(), Reading Digital Inputs, Pull-Down Resistors, Using Internal Pull-Up, Contact Bounce Elimination. Interfacing Analog Sensors: Analog and Digital Signals, Converting an Analog Signal to Digital, ADC Resolution and Quantization, Reading Analog Sensors using analogRead(), Using Analog Inputs to Control Analog Outputs.

TO WATCH VIDEO CLASSES OF THIS COURSE: TAP HERE OR SCAN QR CODE

ARDUINO BASICS

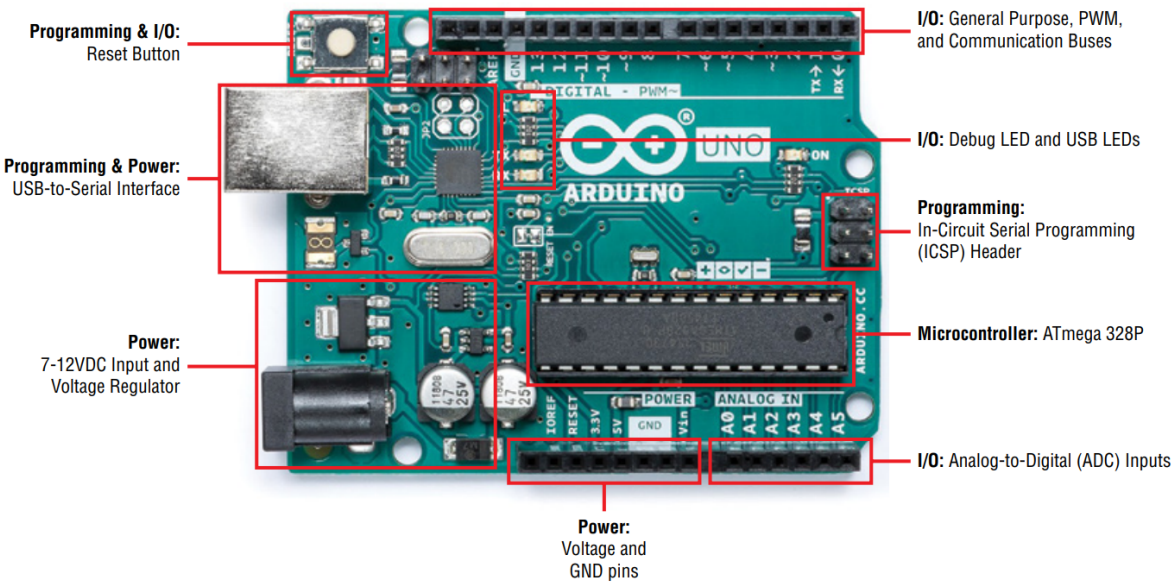
The Arduino is an open-source microcontroller-based platform widely used for building interactive electronic projects. Every Arduino-based system depends on three essential components working together:

- An Arduino board (first-party or third-party compatible)
- External hardware — sensors, actuators, shields, and custom circuits
- The Arduino Integrated Development Environment (IDE) — the software tool used to write, compile, and upload programs

The **Arduino Uno R3** is the standard introductory board and the primary board used throughout this course. It is based on the **ATmega328P** microcontroller and serves as the reference hardware for all Module 1 concepts.

Arduino Uno R3 — Component Overview

The Arduino Uno R3 is organized into four functional groups. Understanding each group is essential for both theoretical questions and practical interfacing.



The figure shows all four functional groups mapped physically onto the board — the MCU sits at the center, the USB interface handles programming, power input is through the barrel jack or USB, and I/O headers run along both edges of the board.

Functional Group	Key Components	Purpose
Microcontroller	ATmega328P	Brain — executes all program instructions
Programming	USB-to-Serial chip, ICSP header, Reset button	Loads programs onto the MCU
Input/Output (I/O)	Digital pins, ADC pins, PWM pins, communication buses	Interfaces with sensors and actuators
Power	DC barrel jack, voltage regulator, 5V/3.3V/GND pins	Supplies and regulates power to the board

The Microcontroller — ATmega328P

The microcontroller unit (MCU) is the brain of every Arduino. It stores compiled code in its program memory and executes instructions sequentially.

The Arduino Uno uses the **ATmega328P**, an 8-bit MCU from Microchip/Atmel, based on the **AVR architecture**. A **16 MHz ceramic resonator** (clock source) is connected to the MCU's clock pins, and every program instruction executes in synchronization with this clock. The MCU's tiny physical pins are connected via copper traces on the PCB to the accessible header pins on the board's edges.

The ATmega328P provides the following peripherals, all accessible through Arduino programming:

- Analog-to-Digital Converters (ADCs) for reading analog sensor voltages
- General-Purpose Input/Output (GPIO) pins for digital interfacing
- Communication buses: I²C, SPI, UART
- PWM-capable output pins
- Serial/USB interface via USART

Microcontroller Architectures in Other Arduino Boards

As applications demand higher performance, Arduino boards are available with different MCU architectures:

Architecture	Bit Width	Example Board	Key Feature
AVR (Atmel)	8-bit	Arduino Uno, Mega 2560	Simple, sufficient for most projects
Arm Cortex-M3	32-bit	Arduino Due	84 MHz, DAC, higher-precision ADC
Arm Cortex-M0	32-bit	Arduino M0/Zero	USB built into MCU
ATmega32U4	8-bit (USB built-in)	Arduino Leonardo, Micro	Can emulate USB keyboard/joystick

Note: An 8-bit architecture represents all addresses and data as 8-bit numbers. A 32-bit architecture uses 32-bit numbers, enabling more memory addressing, faster computation, and richer peripherals.

Microcontrollers in Other Arduino Boards

Arduino Mega 2560

Uses the **ATmega2560** — a supercharged version of the Uno. It offers 54 digital I/O pins, more ADC channels, 4 hardware serial interfaces, and more memory. Suited for projects requiring a large number of connections.

Arduino Leonardo and Micro

Both use the **ATmega32U4**, which has USB functionality built directly into the MCU. No secondary USB-to-serial chip is needed. This enables USB device emulation (keyboard, mouse, joystick).

Arduino Due

Uses a **32-bit Arm Cortex-M3 SAM3X** running at **84 MHz**. Offers higher-precision 12-bit ADCs (0–4095), digital-to-analog converters (DACs), selectable PWM resolution, and a USB host connector. Suitable for music synthesis, complex signal processing, and high-speed applications.

Programming Interfaces

Ordinarily, microcontrollers are programmed in C or assembly using the **In-Circuit Serial Programming (ICSP)** interface with a dedicated hardware programmer. The ICSP header is present on the Uno and is used to program the MCU directly via SPI.

The key innovation of Arduino is that it allows **programming over a standard USB cable**, made possible by the **bootloader**.

USB-to-Serial Interface

On the Arduino Uno, a secondary MCU — the **ATmega16U2** (or ATmega8U2 in older revisions) — acts as a bridge between the USB port and the main MCU's UART serial pins. When you connect the Arduino to a computer, this chip converts USB signals into serial UART signals that the ATmega328P can understand.

On the Arduino Leonardo and M0, USB functionality is built into the main MCU, so no secondary chip is needed.

The Arduino Bootloader and Firmware

A **bootloader** is a small program permanently stored in a reserved section of the MCU's program memory. It enables the Arduino to be programmed over USB without requiring an external hardware programmer.

How the Bootloader Works

When the Arduino powers on or is reset, the bootloader runs first for a brief window (a few seconds).

During this time:

1. It listens on the UART serial interface for an incoming programming command from the IDE.
2. If a command is received, it writes the incoming compiled program into the MCU's main program memory.
3. If no command is received within the timeout window, it immediately jumps to and executes the previously uploaded user sketch.

When you click **Upload** in the Arduino IDE, the IDE instructs the USB-to-serial chip to **reset the main MCU**, forcing it into bootloader mode. Your computer then immediately sends the compiled program over the serial connection, which the bootloader receives and stores.

Fig: AVRISP mkII Programmer — used for direct MCU programming without the bootloader.

Advantages and Limitations of the Bootloader

Aspect	Detail
Advantage	No external programmer required; program via USB
Limitation 1	Occupies approximately 2 KB of program memory
Limitation 2	Adds a few seconds of delay at bootup while checking for upload

If program memory is critical, the bootloader can be removed and the MCU programmed directly via the ICSP header using a programmer or another Arduino acting as a programmer (using "Upload Using Programmer" in the IDE).

Input/Output: GPIO, ADCs, and Communication Buses

The I/O system is the most practically important part of the Arduino for interfacing projects.

GPIO (General-Purpose Input/Output)

All digital pins on the Arduino can function as either digital inputs or digital outputs, configurable in software using pinMode(). Each pin can read or drive signals at logic HIGH (5V) or LOW (0V).

ADC Pins (Analog-to-Digital Converter)

Pins labeled A0–A5 on the Uno are connected to the MCU's built-in 10-bit ADC. These pins measure voltages between 0V and 5V and convert them to integer values from 0 to 1023. These are used for reading analog sensors.

Communication Buses

Many GPIO pins are multiplexed to serve as communication interface pins:

Bus	Pins on Uno	Purpose
UART (Serial)	Pins 0 (RX), 1 (TX)	Asynchronous serial communication
SPI	Pins 10–13	High-speed synchronous serial (sensors, displays)
I^2C	Pins A4 (SDA), A5 (SCL)	Two-wire protocol for multiple devices

PWM Pins

Pins marked with ~ on the Uno (3, 5, 6, 9, 10, 11) support Pulse-Width Modulation — a technique to emulate analog output using digital signals.

Power Supply

The Arduino Uno can be powered in the following ways:

- **USB (5V):** Most convenient during development; powers both the board and logic.
- **DC Barrel Jack:** Accepts 7–12V DC (supports up to 20V, but 7–12V is recommended). An onboard voltage regulator steps this down to 5V.
- **VIN Pin:** Direct voltage input, same range as barrel jack.

The Uno operates at **5V logic levels**. A 3.3V regulated output pin is also available for interfacing with 3.3V peripherals.

The Arduino IDE and First Program

Setting Up

1. Download the Arduino IDE from arduino.cc.
2. Connect the Arduino Uno to the computer via USB.
3. In the IDE, go to **Tools → Board** and select "Arduino Uno."
4. Go to **Tools → Serial Port** and select the correct COM port (Windows: COM*; Linux/macOS: /dev/tty.usbmodem*).
5. Click the **Upload** button to compile and transfer the program. The TX/RX LEDs will flash during upload.

Structure of an Arduino Program (Sketch)

Every Arduino sketch has exactly two mandatory functions:

```
void setup() {
  // Runs ONCE at start — used for pin configuration,
  // initializing communication, etc.
}

void loop() {
  // Runs REPEATEDLY as long as Arduino is powered
}
```

Eg: The Blink Program

```
// Blink: Turns onboard LED on and off every second

void setup() {
  pinMode(LED_BUILTIN, OUTPUT); // Set pin 13 as output
  // LED_BUILTIN resolves to pin 13 on the Uno
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // Turn LED ON (5V)
  delay(1000);                     // Wait 1000 ms (1 second)
  digitalWrite(LED_BUILTIN, LOW);  // Turn LED OFF (0V)
  delay(1000);                     // Wait 1000 ms
}
```

Key functions introduced:

Function	Purpose
pinMode(pin, mode)	Sets pin direction: INPUT or OUTPUT
digitalWrite(pin, value)	Sets digital pin to HIGH (5V) or LOW (0V)
delay(ms)	Pauses execution for specified milliseconds
LED_BUILTIN	Compiler keyword for onboard LED pin (pin 13 on Uno)

const int is used to define pin numbers as read-only constants. This prevents accidental modification and improves code readability.

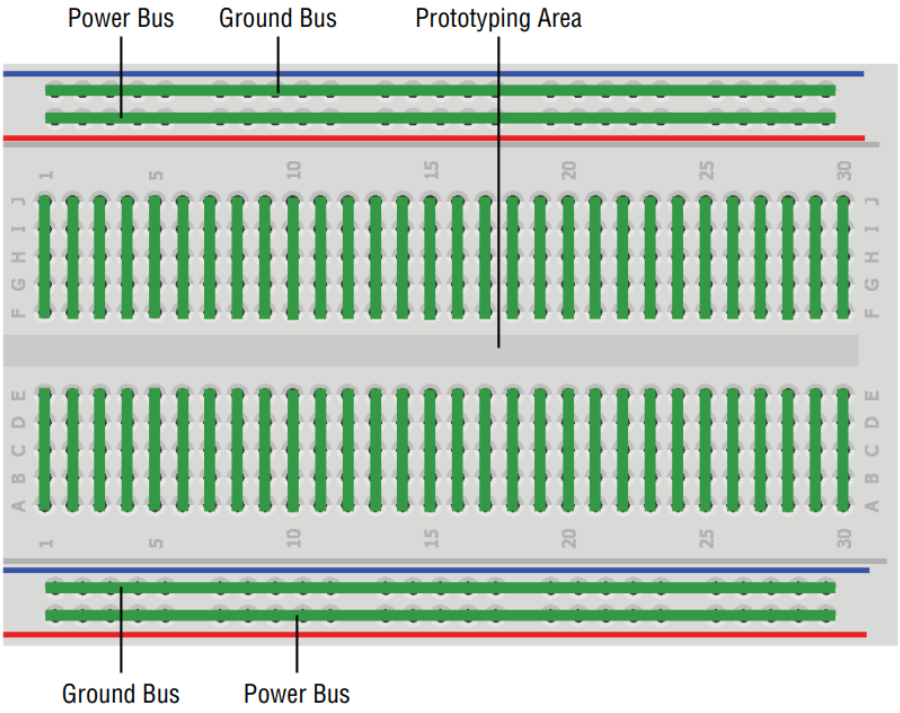
DIGITAL INPUTS, OUTPUTS AND PWM

Breadboards and LED Wiring

Before writing any digital output code, it is essential to understand the hardware setup used for all digital interfacing experiments.

How a Breadboard Works

A breadboard is a solderless prototyping tool that allows temporary circuit connections. Its internal connectivity follows a fixed pattern:



- The **red (+) and blue (−) rails** running along the long edges are power and ground buses respectively. All holes in the red rail are internally connected; same for the blue rail.
- The **vertical columns** in the prototyping area are internally connected in groups of 5, with a **center divider** that separates the two halves — ideal for mounting ICs in DIP package.

Important: The top red/blue buses are NOT internally connected to the bottom red/blue buses. If both sets are needed at the same voltage, they must be bridged with a wire.

LED Polarity and Current Limiting

LED Polarity

An LED (Light Emitting Diode) is a **polarized** component — it must be connected in the correct direction:

- **Anode (+):** Longer lead; connected toward the positive voltage source.
- **Cathode (−):** Shorter lead; flat side on the LED casing; connected toward ground. Current flows from anode to cathode only.

Current Limiting Resistor — Why It Is Mandatory

LEDs have a maximum current rating (typically **20 mA** for standard low-power LEDs). Connecting an LED directly to a 5V pin without a resistor would allow excessive current, potentially burning the LED or damaging the Arduino's output pin. A **resistor in series** limits this current.

Resistor Calculation

Ohm's Law defines the relationship between voltage, current, and resistance: **V = I × R** where V = voltage (volts), I = current (amps), R = resistance (ohms).

Given:

- Supply voltage: 5V
- LED forward voltage drop: 2V
- Desired current: 20 mA = 0.02 A
- Voltage across resistor: 5V − 2V = 3V

Applying Ohm's Law: **R = V / I = 3 / 0.02 = 150 Ω**

A standard **220 Ω** resistor is commonly used — slightly above 150 Ω, it allows sufficient brightness while staying within safe limits.

Power Dissipation Check

The power equation: **P = I × V**
For the resistor: P = 0.02 A × 3V = **60 mW**
A standard through-hole resistor is rated at 125 mW (1/8 W). Since 60 mW < 125 mW, the resistor is safely within its operating limits.

Digital Outputs — Programming

By default, all Arduino pins are configured as **inputs**. To drive an output (such as an LED), the pin must first be explicitly set as an output using pinMode().

Basic Digital Output — Turning an LED On

```
const int LED = 9; // LED connected to pin 9

void setup() {
  pinMode(LED, OUTPUT);     // Set pin 9 as output
  digitalWrite(LED, HIGH);  // Set pin 9 to 5V — LED ON
}

void loop() {
  // Nothing in loop; LED stays ON
}
```

const int declares a **read-only integer constant** — the compiler prevents accidental modification. HIGH sets the pin to 5V; LOW sets it to 0V.

Using For Loops — Variable Blink Rates

A for loop is used when a block of code must execute repeatedly with a changing variable. Its syntax is:

for (initialization; condition; increment) { ... }

Example — LED blink with increasing delay:

```
const int LED = 9;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  for (int i = 100; i <= 1000; i = i + 100) {
    digitalWrite(LED, HIGH);
    delay(i);   // ON for i milliseconds
    digitalWrite(LED, LOW);
    delay(i);   // OFF for i milliseconds
  }
  // i starts at 100, increments by 100 each loop,
  // stops when i exceeds 1000
}
```

Each pass through the loop increases the blink period by 100 ms, visibly slowing down the blink rate from fast to slow before repeating.

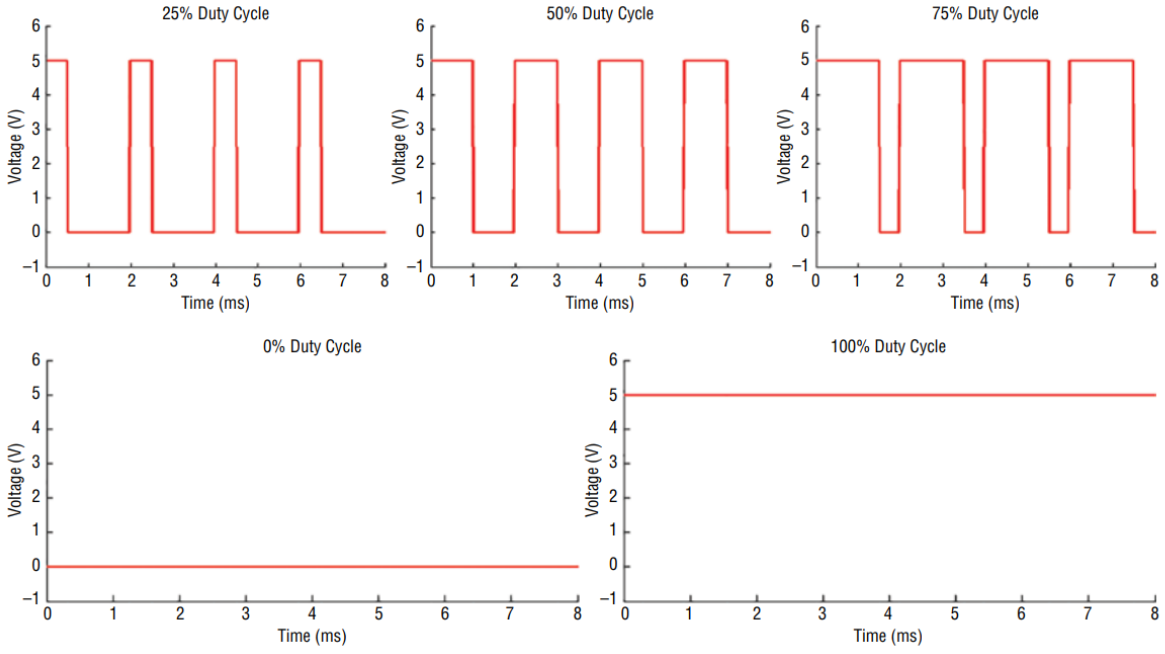
Pulse-Width Modulation (PWM) with analogWrite()

Pulse-Width Modulation (PWM) is a technique where a digital pin rapidly switches between HIGH and LOW to simulate an intermediate analog voltage level. It is not a true analog output but effectively emulates one for many applications such as LED brightness control and motor speed control.

Core Concept — Duty Cycle

PWM works by varying the **duty cycle** of a square wave — the percentage of time the signal remains HIGH in one complete cycle.

Duty Cycle = (Time HIGH / Total Period) × 100%



The average voltage delivered to the load equals (Duty Cycle / 100) × 5V. A 50% duty cycle delivers an effective average of 2.5V, making an LED appear at half brightness.

On the Arduino Uno, the PWM frequency is approximately **490 Hz**, meaning the signal cycles 490 times per second. The human eye cannot perceive flickering above ~60 Hz, so the LED appears to glow at a steady intermediate brightness rather than blinking.

Pins **3, 5, 6, 9, 10, and 11** support PWM on the Uno. They are marked with a ~ symbol on the board.

analogWrite() — Syntax and Behavior

```
analogWrite(pin, value);
```

- pin: A PWM-capable pin
- value: An 8-bit integer from **0 to 255**
    - 0 → 0% duty cycle (always LOW — LED fully OFF)
    - 127 → ~50% duty cycle (half brightness)
    - 255 → 100% duty cycle (always HIGH — LED fully ON)

LED Fade Program

```
const int LED = 9;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  // Fade IN: increase brightness from 0 to 255
  for (int i = 0; i < 256; i++) {
    analogWrite(LED, i);
    delay(10);
  }
  // Fade OUT: decrease brightness from 255 to 0
  for (int i = 255; i >= 0; i--) {
    analogWrite(LED, i);
    delay(10);
  }
  // i++ is shorthand for i = i + 1
  // i-- is shorthand for i = i - 1
}
```

This produces a smooth breathing effect — LED fades from off to full brightness and back, repeating indefinitely.

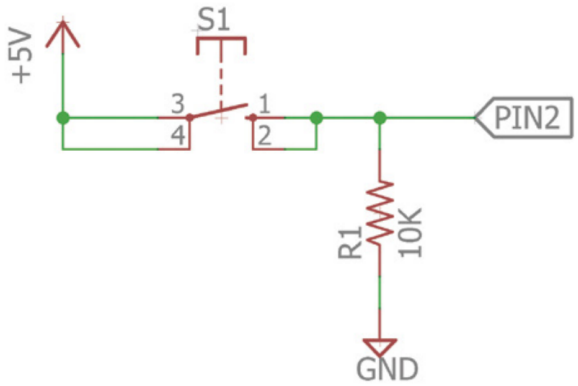
Reading Digital Inputs — Pull-Down Resistors

The Floating Pin Problem

When a digital input pin is not connected to either 5V or GND, it is said to be **floating**. A floating pin picks up electrical noise from nearby signals and gives unpredictable HIGH/LOW readings. This must be prevented.

Pull-Down Resistor

A **pull-down resistor** connects the input pin to GND through a resistor (typically **10 kΩ**). This ensures the pin reads a stable LOW when the button is not pressed.



When the button is open, the pin is weakly pulled to GND through the 10kΩ resistor → reads LOW. When the button is pressed, 5V is directly connected to the pin. The path through the button (near 0 Ω) has far less resistance than the 10kΩ pull-down path. By Ohm's Law, current follows the path of least resistance → pin reads HIGH.

Pull-Up Resistor (Alternate Configuration)

A **pull-up resistor** connects the input pin to 5V through a resistor. The button connects the pin to GND when pressed. In this configuration:

- Button NOT pressed → pin reads HIGH
- Button pressed → pin reads LOW (inverted logic)

Configuration	Default State	When Button Pressed
Pull-Down (to GND)	LOW	HIGH
Pull-Up (to 5V)	HIGH	LOW

Simple LED Control with Button

```
const int LED = 9;
const int BUTTON = 2;

void setup() {
  pinMode(LED, OUTPUT);
  pinMode(BUTTON, INPUT); // Pins are input by default; shown for clarity
}

void loop() {
  if (digitalRead(BUTTON) == LOW) {
    digitalWrite(LED, LOW);   // Button not pressed → LED OFF
  } else {
    digitalWrite(LED, HIGH);  // Button pressed → LED ON
  }
}
```

digitalRead() returns HIGH (1) if the pin is at 5V, or LOW (0) if at 0V. The == operator compares equality; if/else branches execution based on the result.

Using Internal Pull-Up Resistors

The ATmega328P has **internal pull-up resistors** built into every digital I/O pin. These can be enabled in software, eliminating the need for an external 10kΩ resistor on the breadboard.

To activate the internal pull-up:

```
pinMode(BUTTON, INPUT_PULLUP);
```

With INPUT_PULLUP:

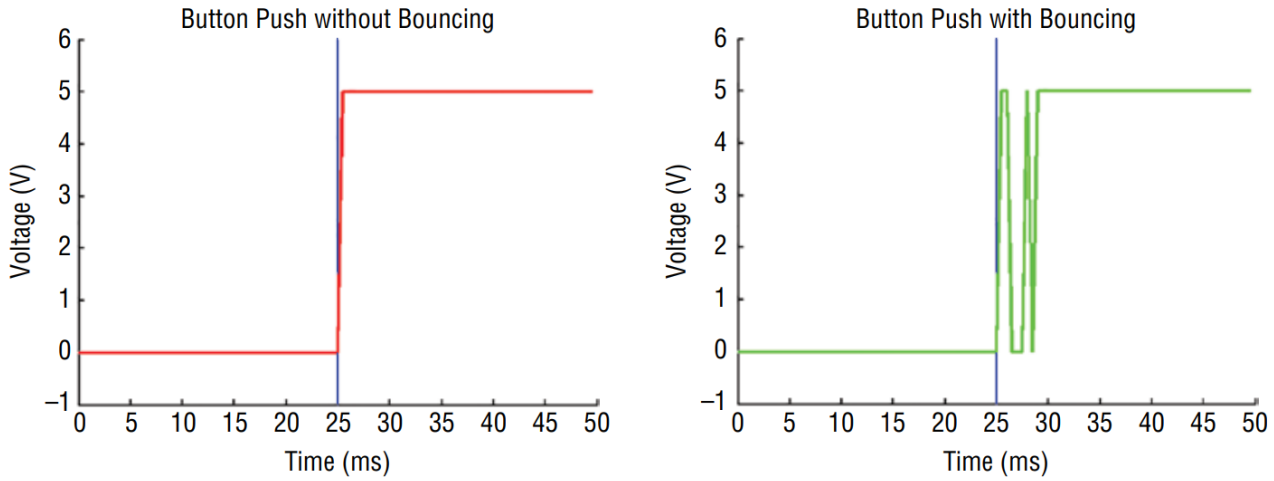
- The pin is internally connected to 5V through a resistor.
- Default state (button open) → pin reads **HIGH**
- Button pressed (connects pin to GND) → pin reads **LOW**

This is the **inverted logic** of the external pull-down configuration. The internal pull-up simplifies circuits and is preferred when building compact designs.

Contact Bounce and Debouncing

The Problem — Switch Bouncing

Mechanical push buttons do not switch cleanly from LOW to HIGH. When pressed, the metal contacts physically bounce — making and breaking contact multiple times over a few milliseconds before settling. This causes the Arduino to register **multiple rapid transitions** instead of one clean press.



Left graph: clean step from 0V to 5V at 25ms. **Right graph:** at 25ms, the signal rapidly oscillates between 0V and 5V several times before settling at 5V at ~30ms.) The left graph shows ideal expected behavior; the right shows actual oscillating bounce. Without debouncing, each bounce is counted as a separate button press.

Software Debouncing — Algorithm

1. Store the previous and current button states (both initialized to LOW).
2. Read the current button state.
3. If current differs from previous → the button has changed state → wait **5 ms** for bouncing to finish.
4. Re-read the button state after 5 ms to get the stable value.
5. If previous was LOW and current is HIGH → button was genuinely pressed → toggle LED.
6. Update previous button state to current.
7. Repeat.

Debounce Program with LED Toggle

```
const int LED = 9;
const int BUTTON = 2;

boolean lastButton = LOW;    // Previous button state (global)
boolean currentButton = LOW; // Current button state (global)
boolean ledOn = false;       // LED on/off state

void setup() {
  pinMode(LED, OUTPUT);
  pinMode(BUTTON, INPUT);
}

/*
 * Debounce Function:
 * Accepts previous button state, returns stable current state.
 */
boolean debounce(boolean last) {
  boolean current = digitalRead(BUTTON); // Read current state
  if (last != current) {   // If state changed (possible bounce)
    delay(5);              // Wait 5ms for bounce to settle
    current = digitalRead(BUTTON); // Re-read stable state
  }
  return current;          // Return stable button state
}

void loop() {
  currentButton = debounce(lastButton); // Get debounced state

  if (lastButton == LOW && currentButton == HIGH) {
    // Button was just pressed (LOW → HIGH transition)
    ledOn = !ledOn;        // Toggle LED state
  }

  lastButton = currentButton; // Update previous state
  digitalWrite(LED, ledOn);   // Apply LED state
}
```

Key Programming Concepts Used:

Concept	Explanation
boolean variable	Holds only true/false (equivalent to HIGH/LOW, 1/0)
Global variables	Declared outside functions; accessible throughout the sketch
!= operator	"Not equal to" — detects state change
&& operator	Logical AND — both conditions must be true
!variable	Logical NOT — inverts the boolean value
User-defined function	debounce() encapsulates the debounce logic for reuse
Return value	return current sends the result back to the caller

Without debouncing: The LED state changes unpredictably when a button is pressed due to multiple bounce transitions being interpreted as multiple presses.

Controlling an RGB LED

RGB LED Structure

An **RGB LED** contains three diodes (Red, Green, Blue) in a single package. A **common-anode** RGB LED has one shared anode pin connected to 5V, and three separate cathode pins — one for each color — controlled individually by the Arduino.

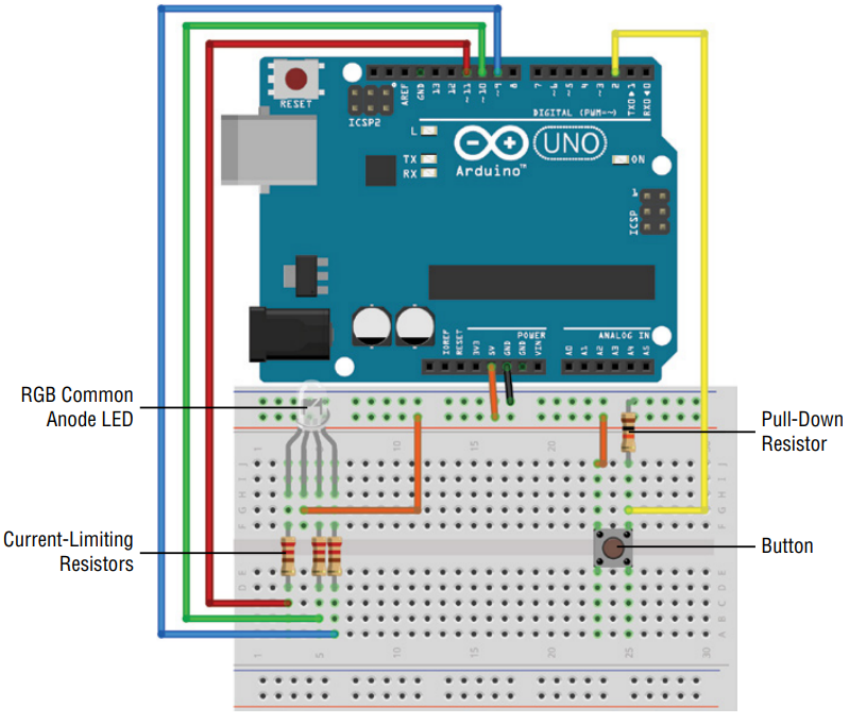
Wiring: Connect the common anode to 5V. Connect each cathode pin (R, G, B) through a **220 Ω resistor** to separate PWM pins (e.g., pins 9, 10, 11).

Inverted Logic for Common-Anode LED

Since the anode is fixed at 5V:

- Setting a cathode pin **LOW** → current flows → that color **turns ON**
- Setting a cathode pin **HIGH** → no potential difference → that color **turns OFF**

This is the **opposite** of a standard LED where HIGH = ON.



RGB Color Mixing with analogWrite()

By varying the PWM value (0–255) on each cathode pin independently, any color can be mixed. Because the logic is **inverted** for common-anode:

- analogWrite(RLED, 0) → Red fully ON
- analogWrite(RLED, 255) → Red fully OFF
- analogWrite(RLED, 127) → Red at half brightness

RGB Nightlight — Color Cycling Program

```
const int BLED = 9;    // Blue cathode → pin 9
const int GLED = 10;   // Green cathode → pin 10
const int RLED = 11;   // Red cathode → pin 11
const int BUTTON = 2;  // Push button → pin 2
```

```
boolean lastButton = LOW;
boolean currentButton = LOW;
int ledMode = 0;       // Tracks current color state (0–7)

void setup() {
  pinMode(BLED, OUTPUT);
  pinMode(GLED, OUTPUT);
  pinMode(RLED, OUTPUT);
  pinMode(BUTTON, INPUT);
}

// Debounce function
boolean debounce(boolean last) {
  boolean current = digitalRead(BUTTON);
  if (last != current) {
    delay(5);
    current = digitalRead(BUTTON);
  }
  return current;
}

// Sets RGB LED color based on mode number
void setMode(int mode) {
  if (mode == 1) {                    // RED
    digitalWrite(RLED, LOW);
    digitalWrite(GLED, HIGH);
    digitalWrite(BLED, HIGH);
  } else if (mode == 2) {             // GREEN
    digitalWrite(RLED, HIGH);
    digitalWrite(GLED, LOW);
    digitalWrite(BLED, HIGH);
  } else if (mode == 3) {             // BLUE
    digitalWrite(RLED, HIGH);
    digitalWrite(GLED, HIGH);
    digitalWrite(BLED, LOW);
  } else if (mode == 4) {             // PURPLE (R+B)
    analogWrite(RLED, 127);
    analogWrite(GLED, 255);
    analogWrite(BLED, 127);
  } else if (mode == 5) {             // TEAL (G+B)
    analogWrite(RLED, 255);
    analogWrite(GLED, 127);
    analogWrite(BLED, 127);
  } else if (mode == 6) {             // ORANGE (R+G)
    analogWrite(RLED, 127);
    analogWrite(GLED, 127);
    analogWrite(BLED, 255);
  } else if (mode == 7) {             // WHITE (R+G+B)
    analogWrite(RLED, 170);
    analogWrite(GLED, 170);
    analogWrite(BLED, 170);
  } else {                            // OFF (mode = 0)
    digitalWrite(RLED, LOW);
    digitalWrite(GLED, LOW);
    digitalWrite(BLED, LOW);
  }
```

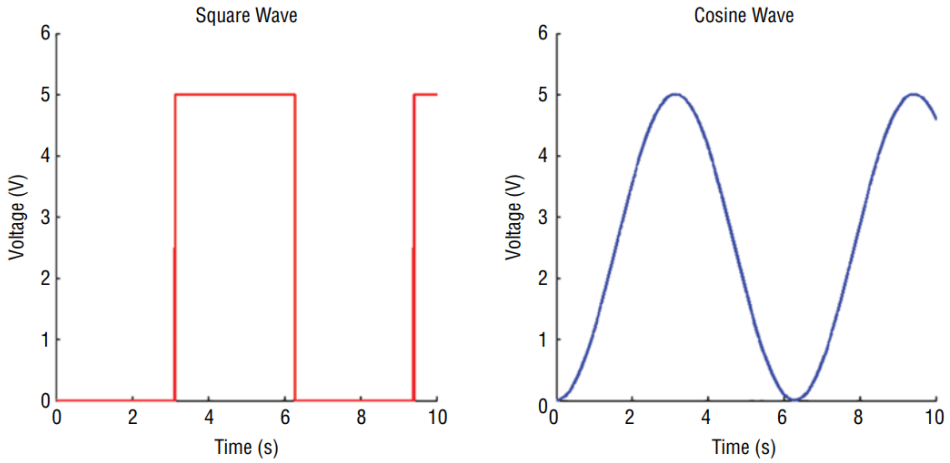
```
}

void loop() {
  currentButton = debounce(lastButton);
  if (lastButton == LOW && currentButton == HIGH) {
    ledMode++;              // Advance to next color
  }
  lastButton = currentButton;
  if (ledMode == 8) ledMode = 0;  // Reset after 7 colors + OFF
  setMode(ledMode);
}
```

INTERFACING ANALOG SENSORS

Analog and Digital Signals

An **analog signal** is one that varies continuously within a range, theoretically taking on an infinite number of values. A **digital signal** is one that exists in only two discrete states — HIGH (5V) or LOW (0V).

The physical world is fundamentally analog. Temperature, light intensity, pressure, and sound are all quantities that vary smoothly and continuously — they cannot be meaningfully represented as just ON or OFF. To process such real-world data with a microcontroller, analog signals must be converted into digital values.



Left graph: a square wave switching cleanly between 0V and 5V — label "Digital Signal". **Right graph:** a smooth cosine/sine wave varying continuously between 0V and 5V — label "Analog Signal".) The digital signal has only two levels; the analog signal takes on every possible value between the two extremes continuously.

Feature	Digital Signal	Analog Signal
Possible values	Only 2 (HIGH / LOW)	Infinite within a range
Example	Button press, LED state	Temperature, light, sound
Arduino handling	digitalRead() / digitalWrite()	analogRead() / analogWrite()

Converting an Analog Signal to Digital — The ADC

Why Conversion is Needed

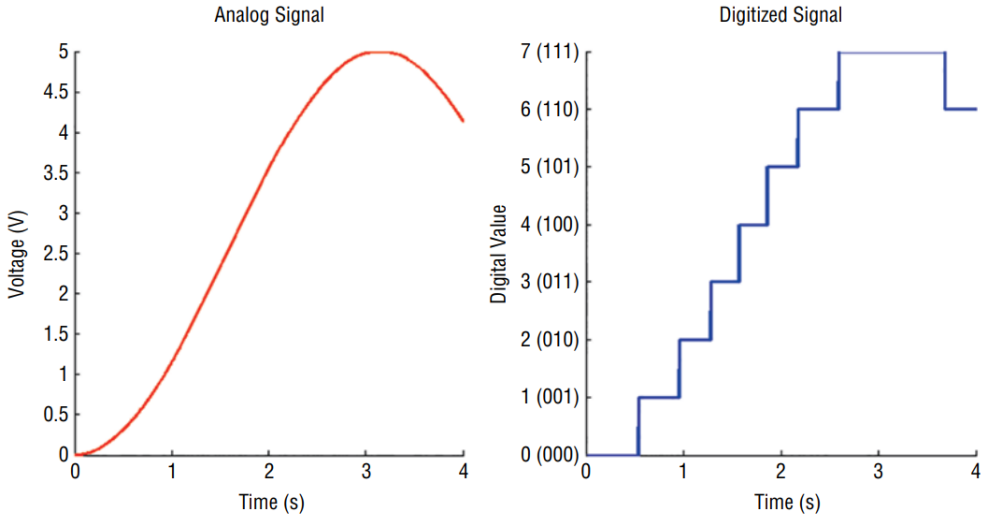
A microcontroller can only process binary (digital) data. To read an analog sensor voltage, it must be converted into a discrete integer value using an **Analog-to-Digital Converter (ADC)**.

ADC Resolution and Quantization

Resolution refers to the number of discrete steps an ADC can divide the input range into. It is expressed in bits.

- An **n-bit ADC** produces 2^n discrete steps.
- The Arduino Uno has a **10-bit ADC**, giving $2^{10} = 1024$ steps (values 0 to 1023).
- The default reference voltage is **5V**, so the ADC maps the range 0V–5V onto 0–1023.

Quantization is the process of assigning a discrete digital value to each sampled analog voltage level. Any analog value is rounded to the nearest available step — this introduces a small error called **quantization error**, which decreases as resolution increases.



Right: the same curve but rendered as a staircase with 8 steps (0–7), showing how the smooth curve is approximated by discrete levels. Label each step 0(000) through 7(111).) The staircase on the right shows quantization — each analog voltage is assigned the nearest digital level. With only 3 bits (8 levels), the approximation is coarse. The Arduino Uno's 10-bit ADC has 1024 steps, making the approximation far more precise.

ADC Voltage-to-Value Mapping (Arduino Uno)

Input Voltage	ADC Output Value
0V	0
2.5V	512
5V	1023

The mapping is linear: **ADC Value = (Input Voltage / Reference Voltage) × 1023**

ADC Resolution Comparison Across Arduino Boards

Board	ADC Resolution	Steps	Notes
Arduino Uno	10-bit	0–1023	Default 5V reference
Arduino Due / Zero	12-bit	0–4095	Higher precision
External ADC (I²C/SPI)	Up to 24-bit	Up to 16M+	For precision instrumentation

Reading Analog Sensors — analogRead()

Syntax: int value = analogRead(pin);

- pin: The analog pin number (0–5 on the Uno, or A0–A5)
- Returns an integer from **0 to 1023**

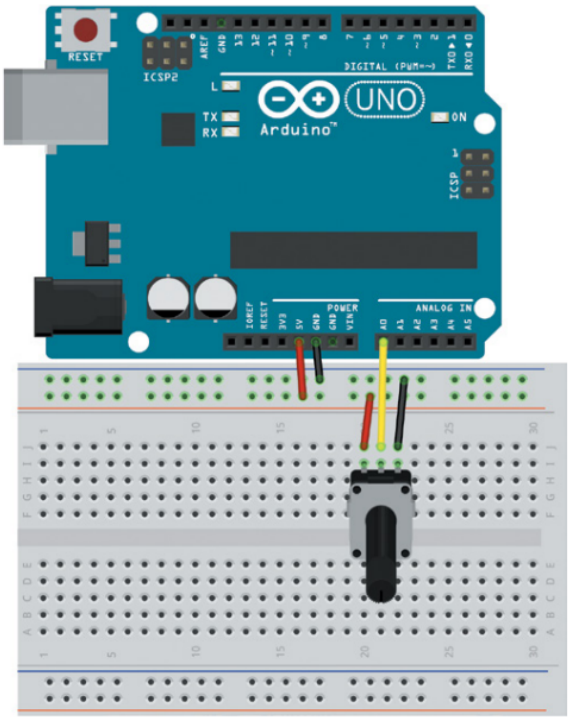
No pinMode() call is required for analog input pins — they are analog inputs by default.

Reading a Potentiometer

A **potentiometer** is a variable resistor with three terminals. It acts as an adjustable voltage divider:

- Outer pin 1 → GND
- Outer pin 2 → 5V
- Middle pin (wiper) → Arduino analog input (A0)

As the knob is turned, the wiper voltage varies between 0V and 5V, and the ADC value varies between 0 and 1023 accordingly.



Displaying ADC Values via Serial Monitor

The **Serial Monitor** in the Arduino IDE displays data sent from the Arduino over USB. It is an essential debugging and calibration tool.

```
const int POT = 0;   // Potentiometer on Analog Pin 0
int val = 0;         // Variable to store ADC reading

void setup() {
  Serial.begin(9600); // Initialize serial at 9600 baud
}

void loop() {
  val = analogRead(POT);    // Read ADC value (0–1023)
  Serial.println(val);      // Send value to Serial Monitor
  delay(500);               // Wait 500ms before next reading
}
```

Key points:

- Serial.begin(9600) sets the communication speed to **9600 baud** (bits per second). The Serial Monitor must be set to the same baud rate.
- Serial.println() sends the value followed by a newline character.
- The TX LED on the Arduino flashes each time data is transmitted.

Using Packaged Analog Sensors

Most analog sensors follow the same three-pin interface as the potentiometer:

- **VCC** → 5V (or 3.3V depending on sensor)
- **GND** → GND
- **Output** → Arduino analog input pin

The sensor internally converts a physical quantity (temperature, distance, acceleration) into a proportional output voltage between 0V and VCC.

Common Analog Sensors

Sensor	Physical Quantity	Output Behavior
TMP36	Temperature	0V at −50°C, 1.75V at 125°C (linear)
Sharp IR Sensor	Distance to object	Voltage decreases as distance increases
Photoresistor (LDR)	Light intensity	Resistance decreases as light increases
ADXL335 Accelerometer	Acceleration (X, Y, Z)	One analog output pin per axis

TMP36 Temperature Sensor — Detail

The TMP36 outputs a voltage linearly proportional to temperature.

The conversion formula is: **Temperature (°C) = (Output Voltage in mV − 500) / 10**

Equivalently: **Output Voltage (mV) = (Temperature °C × 10) + 500**

Example: At 20°C → Voltage = (20 × 10) + 500 = 700 mV. With 5V reference and 10-bit ADC:
ADC Value = (0.700V / 5V) × 1023 ≈ 143

Voltage Dividers and Resistive Sensors

Why a Voltage Divider is Needed

Devices like photoresistors and thermistors **change resistance** in response to physical input, but the Arduino ADC measures **voltage**, not resistance. A **voltage divider** circuit converts the resistance change into a measurable voltage change.

Voltage Divider — Theory

The junction node connects to Arduino A0. Label Vout at the junction.) The output voltage is taken at the junction between R1 and R2. As R1 or R2 changes in value, Vout changes proportionally.

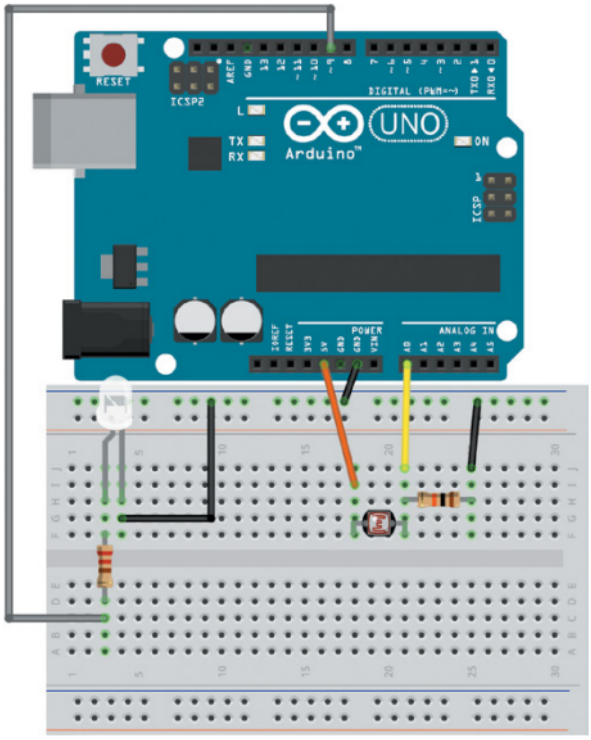
The voltage divider formula: **Vout = Vin × R2 / (R1 + R2)**

Example: If R1 = R2 = 10kΩ and Vin = 5V:
Vout = 5 × 10k / (10k + 10k) = 5 × 0.5 = 2.5V

Photoresistor (LDR) Sensor

A **photoresistor** (Light Dependent Resistor / LDR) changes resistance with light:

- In complete darkness: ~200 kΩ
- In bright light: ~5 kΩ



R1 is the photoresistor (variable). As light increases, R1 decreases, Vout at the junction increases. In darkness, R1 is large, pulling Vout lower. The 10kΩ fixed resistor sets the sensitivity of the divider. The LED on pin 9 will be controlled based on the light reading.

Using Analog Inputs to Control Analog Outputs

The Mapping Problem

analogRead() returns values from **0 to 1023** (10-bit ADC). analogWrite() accepts values from **0 to 255** (8-bit PWM). These ranges do not match directly. Additionally, the practical range of a photoresistor reading may not use the full 0–1023 range.
For example, a room's light level may vary between ADC values of **200** (dark) and **900** (bright).

Two Arduino functions solve this cleanly.

1. map() Function

```
output = map(value, fromLow, fromHigh, toLow, toHigh);
```

Linearly maps value from one range to another.

Example: val = map(val, 200, 900, 255, 0);
This maps:

- ADC value 200 (dark room) → PWM value 255 (LED full brightness)
- ADC value 900 (bright room) → PWM value 0 (LED off)
- ADC value 550 (midpoint) → PWM value 127 (half brightness)

Notice the toLow and toHigh are **swapped (255, 0)** — this **inverts the mapping** so that a darker room produces a brighter LED. The map() function handles inverted mapping automatically.

Important: map() performs linear scaling only. It does **not** constrain the output — if the input exceeds the specified fromLow/fromHigh range, the output may fall outside toLow/toHigh bounds.

2. constrain() Function

output = constrain(value, min, max);

Clamps the value so it never falls below min or exceeds max.

val = constrain(val, 0, 255);

Any mapped value below 0 is clamped to 0; any value above 255 is clamped to 255. This ensures analogWrite() always receives a valid 8-bit input.

Typical Usage — map() followed by constrain()

```
val = analogRead(LIGHT);              // Read sensor: 0–1023
val = map(val, 200, 900, 255, 0);     // Remap to PWM range, inverted
val = constrain(val, 0, 255);         // Clamp within valid PWM range
analogWrite(WLED, val);               // Drive LED with mapped value
```

Complete Programs — Analog Sensor Applications

Program 1 — Temperature Alert System (RGB LED)

Hardware: TMP36 temperature sensor on A0; common-anode RGB LED on pins 9 (Blue), 10 (Green), 11 (Red).
Logic:

- Below lower threshold → LED Blue (too cold)
- Above upper threshold → LED Red (too hot)
- Within range → LED Green (normal)

```
const int BLED = 9;           // Blue LED cathode → pin 9
const int GLED = 10;          // Green LED cathode → pin 10
const int RLED = 11;          // Red LED cathode → pin 11
const int TEMP = 0;           // TMP36 output → analog pin A0
const int LOWER_BOUND = 139;  // ADC value for ~18°C
const int UPPER_BOUND = 147;  // ADC value for ~22°C
int val = 0;

void setup() {
  pinMode(BLED, OUTPUT);
  pinMode(GLED, OUTPUT);
  pinMode(RLED, OUTPUT);
}
```

```
void loop() {
  val = analogRead(TEMP);       // Read temperature sensor

  if (val < LOWER_BOUND) {      // Too cold → Blue
    digitalWrite(RLED, HIGH);   // Red OFF
    digitalWrite(GLED, HIGH);   // Green OFF
    digitalWrite(BLED, LOW);    // Blue ON (common anode: LOW = ON)
  }
  else if (val > UPPER_BOUND) { // Too hot → Red
    digitalWrite(RLED, LOW);    // Red ON
    digitalWrite(GLED, HIGH);   // Green OFF
    digitalWrite(BLED, HIGH);   // Blue OFF
  }
  else {                        // Normal → Green
    digitalWrite(RLED, HIGH);   // Red OFF
    digitalWrite(GLED, LOW);    // Green ON
    digitalWrite(BLED, HIGH);   // Blue OFF
  }
}
```

Threshold Derivation:

Using: **Voltage (mV) = (Temp°C × 10) + 500**

Temperature	Voltage	ADC Value
18°C	680 mV	139
20°C	700 mV	143
22°C	720 mV	147

ADC Value = (Voltage / 5000) × 1023
For calibration: upload the analogRead() + Serial.println() sketch first, observe values at known temperatures, and set thresholds accordingly.

Program 2 — Automatic Nightlight (Photoresistor + White LED)

Hardware: Photoresistor + 10kΩ fixed resistor voltage divider on A0; white LED (anode) on pin 9 (PWM).
Logic: As room gets darker (ADC value decreases), LED gets brighter. As room brightens (ADC value increases), LED dims.

```
const int WLED = 9;         // White LED anode → pin 9 (PWM)
const int LIGHT = 0;        // Photoresistor divider → analog pin A0
const int MIN_LIGHT = 200;  // ADC value at darkest expected condition
const int MAX_LIGHT = 900;  // ADC value at brightest expected condition
int val = 0;

void setup() {
  pinMode(WLED, OUTPUT);
}
```

```
void loop() {
  val = analogRead(LIGHT);              // Read light sensor
  val = map(val, MIN_LIGHT, MAX_LIGHT,
            255, 0);            // Dark(200)→255, Bright(900)→0
  val = constrain(val, 0, 255);       // Clamp to valid PWM range
  analogWrite(WLED, val);             // Set LED brightness
}
```

MIN_LIGHT and MAX_LIGHT must be calibrated using the Serial Monitor for each specific photoresistor and lighting environment. The map() function inverted mapping (255, 0) ensures the nightlight brightens in darkness.

MODEL QUESTIONS

1. What is the function of the bootloader in Arduino ? (3 Marks)
2. Design the value of resistance required to connect an LED to the digital output pin of an Arduino. (3 Marks)
3. Write code for a temperature alert system where the color of an RGB LED will indicate whether the temperature is too low, normal or too high. (4 Marks)
4. What is the difference between map() and constrain() functions? Mention typical use of these functions. (5 Marks)
5. Explain the principle behind the analogwrite() function. Write code to vary the brightness of an LED using analogwrite(). (4 Marks)
6. With a circuit diagram and code, explain how a push button connected to a digital output pin can be used to turn on/off an LED connected to a digital output pin. (5 Marks)
